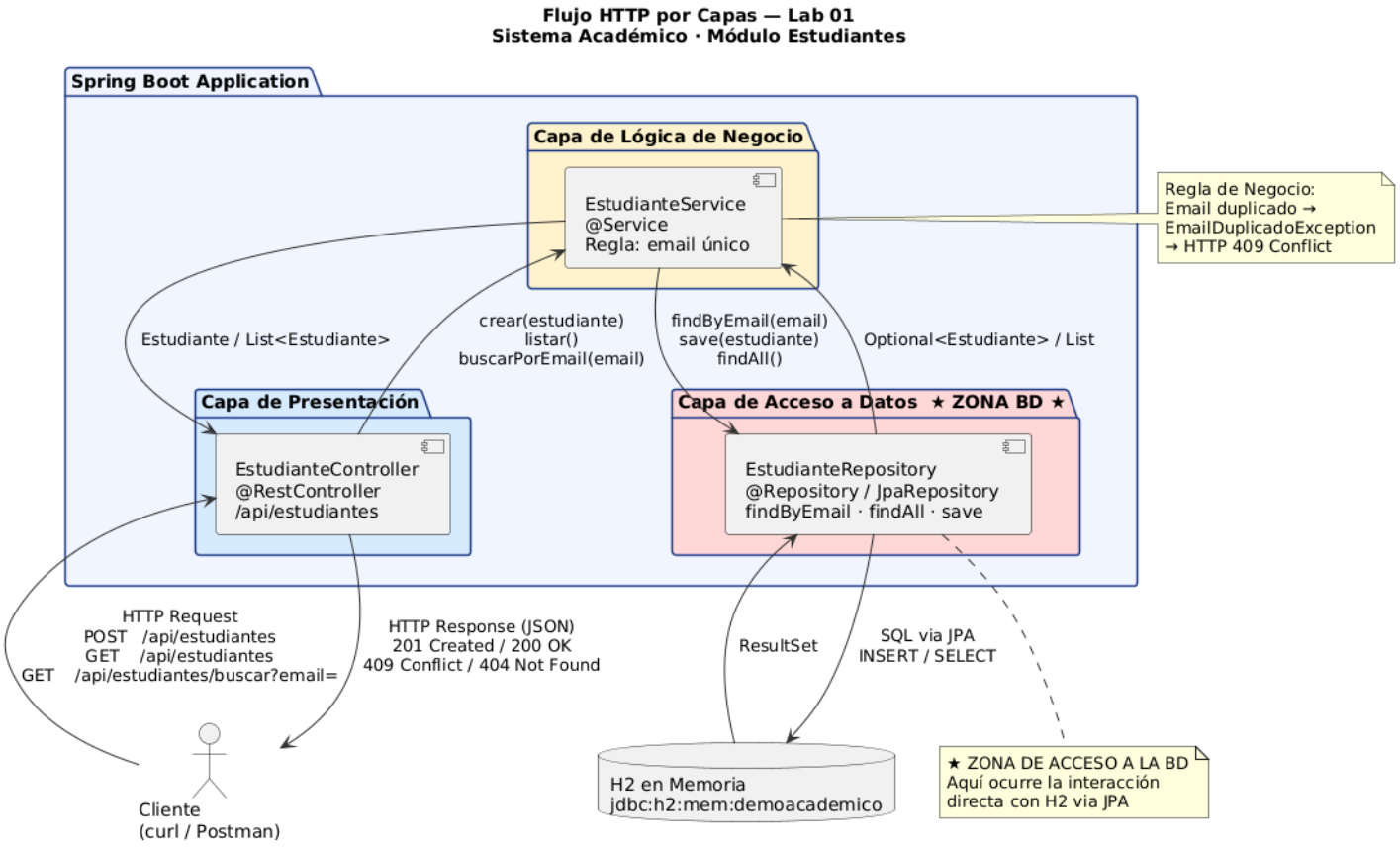
Entregable Lab 01 — Ingeniería de Software III

2191918 Kevin Andres Gallardo Robles

2155160 Cristian Eduardo Johnson Acevedo

2162643 Johan Sebastián Villamizar Ortega

1. Diagrama de flujo por capas



2. Estructura del proyecto por capas

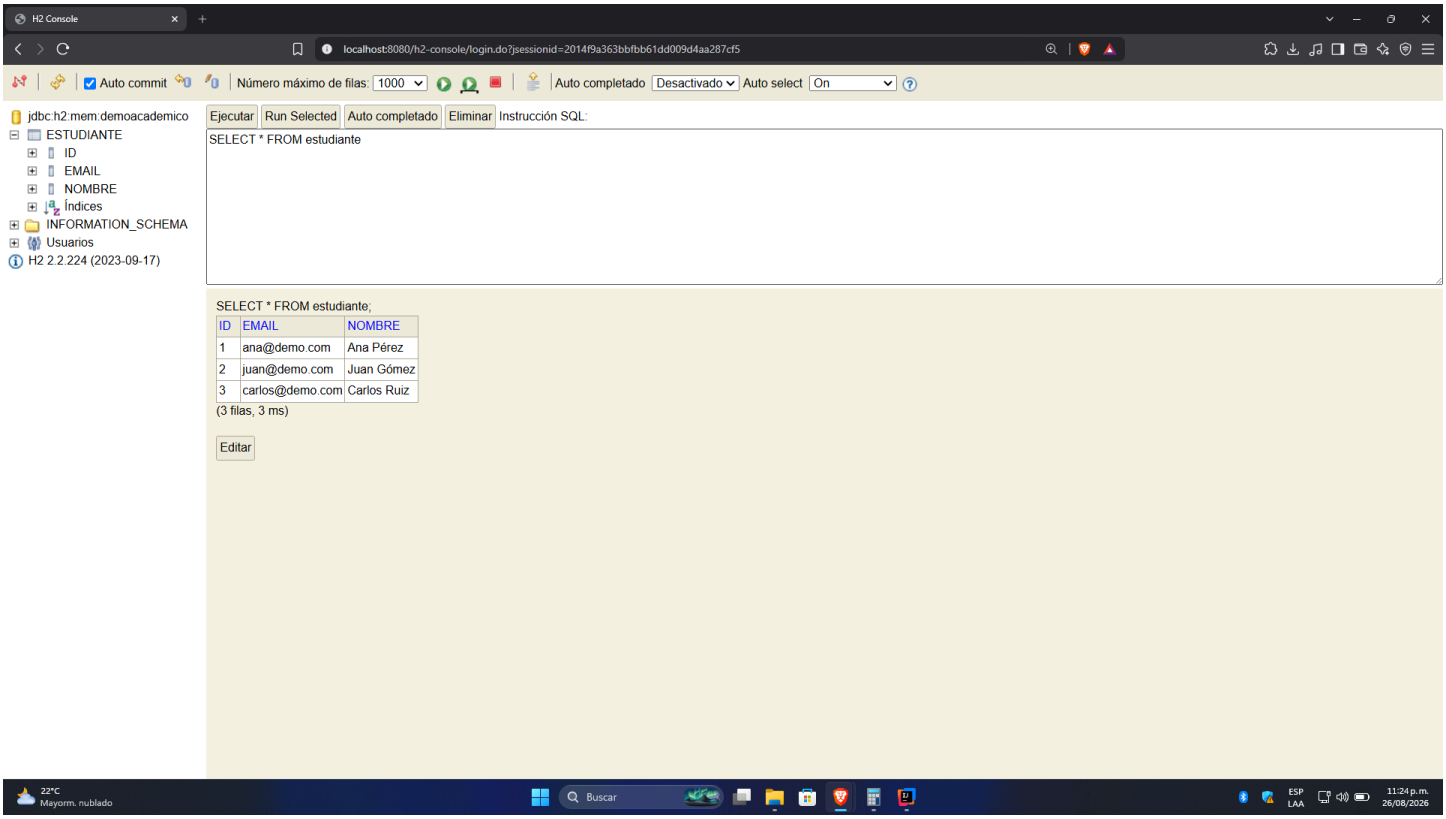
Capa	Clase	Responsabilidad
Presentación	EstudianteController	Recibe peticiones HTTP, retorna JSON. Sin lógica de negocio.
Lógica de Negocio	EstudianteService	Aplica reglas (email único). No conoce HTTP ni SQL directo.

Capa	Clase	Responsabilidad
Acceso a Datos	EstudianteRepository	Interacción directa con H2. Genera SQL automáticamente via JPA.
Modelo	Estudiante	Entidad JPA. Mapea la tabla estudiante en H2.

3. Endpoints implementados

Método	URL	Descripción	Respuesta
POST	/api/estudiantes	Crear estudiante	201 Created / 409 Conflict
GET	/api/estudiantes	Listar todos	200 OK + JSON array
GET	/api/estudiantes/buscar?email=	Buscar por email	200 OK / 404 Not Found

4. Captura de H2 Console



5. Tres limitaciones identificadas del modelo por capas

Limitación 1 — Acoplamiento fuerte entre capas contiguas

El Controller depende directamente del tipo concreto **EstudianteService** y el Service depende de **EstudianteRepository.** Si cambia la firma de un método en una capa, todas las capas superiores deben actualizarse.

```java
// EstudianteController.java
// El controller recibe el Service por constructor, acoplado al tipo concreto
public EstudianteController(EstudianteService service) {
    this.service = service;
}
```

No se usan interfaces intermedias (ej. **IEstudianteService**), por lo que un cambio en **EstudianteService** obliga a recompilar el controller.

Limitación 2 — La entidad de dominio cruza todas las capas

El objeto **Estudiante** (entidad JPA) se usa como DTO de entrada en el controller **(@RequestBody Estudiante**), como entidad de negocio en el service y como objeto de persistencia en el repository. Esto mezcla responsabilidades: un cambio en la tabla de BD impacta directamente el contrato de la API REST.

```java
// EstudianteController.java
// POST /api/estudiantes — crea un estudiante, retorna 409 si el email ya existe
@PostMapping
public ResponseEntity<?> crear(@Valid @RequestBody Estudiante estudiante) {
    // La misma clase @Entity llega directamente desde el JSON del cliente.
    // Si se agrega una columna interna en BD (ej. fechaCreacion),
    // el cliente podría enviarla o recibirla sin control.
    try {
        Estudiante creado = service.crear(estudiante);
        return ResponseEntity.status(HttpStatus.CREATED).body(creado);
    } catch (EstudianteService.EmailDuplicadoException ex) {
        return ResponseEntity
                .status(HttpStatus.CONFLICT)
                .body(Map.of("error", ex.getMessage()));
    }
}
```

Una arquitectura más robusta usaría DTOs separados (**EstudianteRequestDTO**, **EstudianteResponseDTO**).

Limitación 3 — Regla de negocio difícil de testear en aislamiento

La regla de email único en **EstudianteService** llama directamente al repository, lo que hace que cualquier prueba unitaria requiera un mock del repositorio o levantar el contexto completo de Spring.

```java
// EstudianteService.java
public Estudiante crear(Estudiante estudiante) {  1 usage
    // Regla de negocio: el email debe ser único
    // Esta verificación llama al repository → acoplada a la BD
    if (repository.findByEmail(estudiante.getEmail()).isPresent()) {
        throw new EmailDuplicadoException("El email '" + estudiante.getEmail() + "' ya está registrado.");
    }
    return repository.save(estudiante);
}
```

Al crecer el sistema con más reglas de negocio, esta dependencia directa dificulta pruebas unitarias rápidas y aisladas. Una arquitectura hexagonal (ports & adapters) resolvería este problema.